

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

Facultad de Ciencias Fisico Matematicas

GamerRank

Sistema de Rankings y Estadísticas de Jugadores

Producto Integrador de Aprendizaje (PIA)

Programación Estructurada

Integrantes	Información
Integrante 1	Hector Emiliano Garcia Luevano — 2148010 — Grupo 035
Integrante 2	Dariana Michel Guevara Corpus — 2082041 — Grupo 033
Integrante 3	Luis Angel Hernandez Hernandez — 2118162 — Grupo 033
Maestra	Reyna Guadalupe Castro Medellin
Fecha de entrega	9 de mayo de 2026

1. Justificación de Innovación

GamerRank lo pensamos como respuesta a una necesidad dentro del mundo de los videojuegos: la falta de un sistema centralizado, accesible y personalizado que permita a jugadores casuales y competitivos dar seguimiento a su rendimiento a lo largo del tiempo y compararse con otros jugadores.

La innovación del proyecto radica en los siguientes puntos:

- Comparación inteligente contra el promedio global: el sistema calcula automáticamente los promedios de puntaje, winrate y victorias de todos los jugadores registrados y determina si un jugador está por encima o por debajo en cada categoría, generando un diagnóstico de rendimiento personalizado.
- Rankings dinámicos con múltiples criterios: el usuario puede ordenar el ranking por puntaje total, winrate o número de victorias, además de filtrar por país o por juego específico, lo cual va más allá de un sistema de tabla simple.
- Visualización gráfica del progreso: mediante la librería Matplotlib se generan gráficas interactivas que muestran la evolución del jugador a lo largo de sus partidas, comparativas de winrate entre jugadores y rankings visuales.
- Sistema de rangos automático: conforme el jugador acumula puntos, el sistema actualiza su rango (Bronce, Plata, Oro, Platino, Diamante) sin intervención manual, similar a los sistemas de ranked de juegos competitivos modernos.
- Persistencia de datos con acceso directo: todos los datos se almacenan en archivos binarios con acceso directo mediante fseek, lo que permite actualizar registros individuales sin reescribir todo el archivo, mejorando el rendimiento del sistema.

2. Explicación General del Sistema

2.1 Proceso general de funcionamiento

Al iniciar el sistema, GamerRank carga automáticamente todos los jugadores registrados desde el archivo binario jugadores.dat hacia un arreglo en memoria. A continuación solicita al usuario su username y lo busca en la lista cargada.

Si el jugador no existe, el sistema solicita sus datos (país y email) con validación mediante expresiones regulares y lo registra tanto en memoria como en el archivo. Si ya existe, carga sus datos y lo lleva directamente al menú principal.

El menú principal opera con un switch alfanumérico que acepta las letras J (Jugadores), P (Partidas), R (Rankings), E (Estadísticas) y S (Salir), permitiendo al usuario navegar entre los cuatro subgrupos funcionales del sistema.

2.2 Subgrupos del sistema

Subgrupo Jugadores: permite registrar nuevos jugadores, editar sus datos de país y email, eliminar cuentas y buscar jugadores por username. Todas las operaciones se sincronizan con el archivo jugadores.dat mediante acceso directo.

Subgrupo Partidas: permite al jugador activo registrar nuevas partidas ingresando el nombre del juego, puntos obtenidos y resultado (victoria o derrota). El historial se puede consultar y editar. Cada partida registrada actualiza automáticamente las estadísticas del jugador y su rango.

Subgrupo Rankings: ofrece cuatro vistas del ranking: global, por juego, por país y Top 10. En todas se puede elegir el criterio de ordenamiento (puntaje, winrate o victorias). El ordenamiento se realiza mediante el algoritmo de burbuja (bubble sort).

Subgrupo Estadísticas: muestra las estadísticas individuales del jugador activo comparadas contra el promedio global del sistema, genera gráficas con Matplotlib, muestra el rendimiento general de todos los jugadores y permite exportar reportes en formato tabular a archivos .txt secuenciales.

2.3 Datos principales

Estructura	Campo	Descripción
Jugador	username[30]	Identificador único del jugador
Jugador	email[50]	Correo electrónico validado con regex
Jugador	pais[30]	País de origen validado con regex
Estadísticas	puntaje_total	Suma acumulada de puntos de todas las partidas
Estadísticas	winrate	Porcentaje de victorias sobre partidas jugadas
Estadísticas	rango_actual	Rango calculado automáticamente según puntaje
Historial	partidas[50]	Arreglo de hasta 50 partidas por jugador
Partida	juego[40]	Nombre del videojuego
Partida	puntos	Puntos obtenidos en la partida
Partida	victoria	Booleano que indica si fue victoria o derrota

3. Problemas Enfrentados y Soluciones

3.1 Expresiones regulares en Mac con REG_EXTENDED

Problema: al compilar el patrón de validación con cuantificadores {n,m} usando REG_EXTENDED, el compilador generaba el error invalid repetition count(s), impidiendo que la expresión regular compilara correctamente.

Solución: se sustituyó el patrón con cuantificadores numéricos por el operador + combinado con validación de longitud mediante strlen() antes de llamar a regex(). Esto permitió mantener el uso de la librería <regex.h> cumpliendo el requisito del PIA sin depender de cuantificadores no soportados.

3.2 Integración de Matplot++ con CLion en Mac

Problema: al instalar Matplot++ con Homebrew e intentar compilar desde CLion, el compilador no encontraba el archivo de cabecera `matplotlib/matplotlib.h` porque CMake no sabía dónde estaba instalada la librería.

Solución: se configuró el archivo `CMakeLists.txt` agregando la ruta de Homebrew mediante `list(APPEND CMAKE_PREFIX_PATH)` y se enlazó la librería con `target_link_libraries` usando `Matplot++::matplotlib`, lo que permitió compilar y ejecutar las gráficas directamente desde el IDE.

3.3 Mezcla de código C y C++ en el mismo proyecto

Problema: al intentar llamar funciones de `graficas.cpp` (C++) desde `main.c` (C), el enlazador generaba errores de símbolo no encontrado debido a la diferencia en el name mangling entre C y C++.

Solución: se declararon las funciones de gráficas en `graficas.h` usando `extern "C"` dentro de un bloque `#ifdef __cplusplus`, lo que indica al compilador de C++ que exponga esas funciones con nombres compatibles con C, permitiendo la interoperabilidad entre ambos lenguajes.

3.4 Persistencia correcta del historial en archivos binarios

Problema: al registrar un jugador nuevo sin inicializar las estructuras anidadas, los campos de Historial y Estadísticas contenían valores basura en memoria, lo que causaba que al releer el archivo se mostraran datos incorrectos.

Solución: se agregó una llamada a `memset()` sobre las estructuras Estadísticas e Historial inmediatamente antes de escribir el jugador nuevo en el archivo, garantizando que todos los campos comiencen en cero.

4. Aportación de Cada Integrante

Integrante	Aportación al proyecto
Hector Emiliano Garcia Luevano	Diseño de la arquitectura general del sistema y definición de estructuras de datos. Implementación del flujo principal en <code>main.c</code> , validación con expresiones regulares, subgrupo de Jugadores (registro, edición, eliminación, búsqueda) y configuración del entorno de desarrollo con CMake y Matplot++.
Dariana Michel Guevara Corpus	Implementación del subgrupo de Partidas (registro, edición, historial) y del sistema de actualización automática de estadísticas y rangos. Desarrollo de las funciones de validación booleanas y manejo de archivos de acceso directo con <code>fseek</code> y <code>fwrite</code> .
Luis Angel Hernandez Hernandez	Implementación del subgrupo de Rankings con algoritmo bubble sort y filtros por país y juego. Desarrollo del subgrupo de Estadísticas, comparación contra promedio global, generación de reportes en archivos secuenciales y creación de gráficas con Matplot++ en <code>graficas.cpp</code> .

